REFERENCE

CLI reference

Complete reference for Claude Code command-line interface, including commands and flags.

CLI commands

You can start sessions, pipe content, resume conversations, and manage updates with these commands:

Command	Description	Example
`claude`	Start interactive session	`claude`
`claude "query"`	Start interactive session with initial prompt	`claude "explain this project"`
`claude -p "query"`	Query via SDK, then exit	`claude -p "explain this function"`
`cat file | claude -p "query"`	Process piped content	`cat logs.txt | claude -p "explain"`
`claude -c`	Continue most recent conversation in current directory	`claude -c`
`claude -c -p "query"`	Continue via SDK	`claude -c -p "Check for type errors"`
`claude -r "<session>" "query"`	Resume session by ID or name	`claude -r "auth-refactor" "Finish this PR"`
`claude update`	Update to latest version	`claude update`
`claude install [version]`	Install or reinstall the native binary. Accepts a version like `2.1.118`, or `stable` or `latest`. See **Install a specific version**	`claude install stable`
`claude auth login`	Sign in to your Anthropic account. Use `--email` to pre-fill your email address, `--sso` to force SSO authentication, and `--console` to sign in with Anthropic Console for API usage billing instead of a Claude subscription	`claude auth login --console`
`claude auth logout`	Log out from your Anthropic account	`claude auth logout`
`claude auth status`	Show authentication status as JSON. Use `--text` for human-	`claude auth status`

Command	Description	Example
	readable output. Exits with code 0 if logged in, 1 if not	
`claude agents`	Open **agent view** to monitor and dispatch parallel background sessions. Use `--cwd <path>` to show only sessions started under that directory, or `--json` to print active sessions as a JSON array for scripting (`--json --all` also includes completed background sessions). Pass `--permission-mode` , `--model` , `--effort` , or `--agent` to set **defaults for dispatched sessions**. Accepts `--settings` , `--add-dir` , `--plugin-dir` , and `--mcp-config` like the top-level `claude` command. Opening agent view requires an interactive terminal	`claude agents --json`
`claude attach <id>`	Attach to a **background session** in this terminal	`claude attach 7c5dcf5d`
`claude auto-mode defaults`	Print the built-in **auto mode** classifier rules as JSON. Use `claude auto-mode config` to see your effective config with settings applied	`claude auto-mode defaults > rules.json`
`claude daemon status`	Print the background-session **supervisor's** state, version, socket directory, and worker count for diagnostics. Exits 1 if the supervisor isn't running	`claude daemon status`
`claude daemon stop --any`	Stop the background-session **supervisor** and the sessions it hosts. Pass `--keep-workers` to leave background sessions running so the next supervisor reconnects to them. `--any` confirms stopping an on-demand supervisor, which is the default. Use this to recover from an **unresponsive supervisor**	`claude daemon stop --any --keep-workers`
`claude logs <id>`	Print recent output from a **background session**	`claude logs 7c5dcf5d`
`claude mcp`	Configure Model Context Protocol (MCP) servers	See the **Claude Code MCP documentation**.
`claude mcp login <name>`	Run a configured MCP server's OAuth flow without opening the interactive `/mcp` panel. Works for HTTP, SSE, and claude.ai connector servers. Add `--no-browser` over SSH to print the authorization URL instead of opening a browser, then paste the redirect URL back at the prompt. Requires Claude Code v2.1.186 or later. See **Authenticate from the command line**	`claude mcp login sentry`
`claude mcp logout <name>`	Clear stored OAuth credentials for an MCP server. Requires Claude Code v2.1.186 or later	`claude mcp logout sentry`
`claude plugin`	Manage Claude Code **plugins**. Alias: `claude plugins` . See **plugin reference** for subcommands	`claude plugin install code-review@claude-plugins-official`
`claude project purge [path]`	Delete all local Claude Code state for a project: transcripts, task lists, debug logs, file-edit history, prompt history lines, and the project's entry in `~/.claude.json` . Omit `[path]` to pick from an interactive list. Flags: `--dry-run` to preview, `-`	`claude project purge ~/work/repo --dry-run`

Command	Description	Example
	`y` / `--yes` to skip confirmation, `-i` / `--interactive` to confirm each item, `--all` for every project. See **Clear local data**	
`claude remote-control`	Start a **Remote Control** server to control Claude Code from Claude.ai or the Claude app. Runs in server mode (no local interactive session). See **Server mode flags**	`claude remote-control --name "My Project"`
`claude respawn <id>`	Restart a **background session**, running or stopped, with its conversation intact. Use `--all` to restart every running session, e.g. to pick up an updated Claude Code binary	`claude respawn 7c5dcf5d`
`claude rm <id>`	Remove a **background session** from the list. The conversation transcript stays on your local machine, available through `claude --resume`	`claude rm 7c5dcf5d`
`claude setup-token`	Generate a long-lived OAuth token for CI and scripts. Prints the token to the terminal without saving it. Requires a Claude subscription. See **Generate a long-lived token**	`claude setup-token`
`claude stop <id>`	Stop a **background session**. Also accepts `claude kill`	`claude stop 7c5dcf5d`
`claude ultrareview [target]`	Run **ultrareview** non-interactively. Prints findings to stdout and exits 0 on success or 1 on failure. Use `--json` for the raw payload and `--timeout <minutes>` to override the 30-minute default	`claude ultrareview 1234 --json`

If you mistype a subcommand, Claude Code suggests the closest match and exits without starting a session. For example, `claude udpate` prints `Did you mean claude update?`.

CLI flags

Customize Claude Code's behavior with these command-line flags. `claude --help` does not list every flag, so a flag's absence from `--help` does not mean it is unavailable.

Flag	Description	Example
`--add-dir`	Add additional working directories for Claude to read and edit files. Grants file access; most `.claude/` configuration is **not discovered** from these directories. Validates each path exists as a directory. To persist these directories across sessions, set `permissions.additionalDirectories` in settings	`claude --add-dir ../apps ../lib`
`--advisor <model>`	Enable the server-side **advisor tool** for this session with a model alias: `opus`, `sonnet`, or `fable` (v2.1.170+), or a full model ID. Takes precedence over the `advisorModel` setting for the session. Requires Claude Code v2.1.98 or later	`claude --advisor opus`

Flag	Description	Example
`--agent`	Specify an agent for the current session (overrides the `agent` setting)	`claude --agent my-custom-agent`
`--agents`	Define custom subagents dynamically via JSON. Uses the same field names as subagent **frontmatter**, plus a `prompt` field for the agent's instructions	`claude --agents '{"reviewer": {"description":"Reviews code","prompt":"You are a code reviewer"}}'`
`--allow-dangerously-skip-permissions`	Add `bypassPermissions` to the `Shift+Tab` mode cycle without starting in it. Lets you begin in a different mode like `plan` and switch to `bypassPermissions` later. See **permission modes**	`claude --permission-mode plan --allow-dangerously-skip-permissions`
`--allowedTools`, `--allowed-tools`	Tools that execute without prompting for permission. See **permission rule syntax** for pattern matching. To restrict which tools are available, use `--tools` instead	`"Bash(git log *)" "Bash(git diff *)" "Read"`
`--append-system-prompt`	Append custom text to the end of the default system prompt	`claude --append-system-prompt "Always use TypeScript"`
`--append-system-prompt-file`	Load additional system prompt text from a file and append to the default prompt	`claude --append-system-prompt-file ./extra-rules.txt`
`--ax-screen-reader`	Render screen-reader friendly output: flat text without decorative borders or animations. Forces the classic renderer, so the `tui` setting has no effect for the session. Takes precedence over `CLAUDE_AX_SCREEN_READER` and the `axScreenReader` setting. Requires Claude Code v2.1.181 or later	`claude --ax-screen-reader`
`--bare`	Minimal mode: skip auto-discovery of hooks, skills, plugins, MCP servers, auto memory, and CLAUDE.md so scripted calls start faster. Claude has access to Bash, file read, and file edit tools. Sets `CLAUDE_CODE_SIMPLE`. See **bare mode**	`claude --bare -p "query"`
`--betas`	Beta headers to include in API requests (API key users only)	`claude --betas interleaved-thinking`
`--bg`, `--background`	Start the session as a **background agent** and return immediately. Prints the session ID and management commands. Combine with `--exec` to run a shell command as a background job instead of a Claude session, or with `--agent` to run a specific subagent	`claude --bg "investigate the flaky test"`
`--channels`	(Research preview) MCP servers whose **channel** notifications Claude should listen for in this session. Space-separated list of `plugin:<name>@<marketplace>` entries. Requires Claude.ai authentication	`claude --channels plugin:my-notifier@my-marketplace`

Flag	Description	Example
`--chrome`	Enable **Chrome browser integration** for web automation and testing	`claude --chrome`
`--continue`, `-c`	Load the most recent conversation in the current directory. Includes sessions that added this directory with `/add-dir`	`claude --continue`
`--dangerously-load-development-channels`	Enable **channels** that are not on the approved allowlist, for local development. Accepts `plugin:<name>@<marketplace>` and `server:<name>` entries. Prompts for confirmation	`claude --dangerously-load-development-channels server:webhook`
`--dangerously-skip-permissions`	Skip permission prompts. Equivalent to `--permission-mode bypassPermissions`. See **permission modes** for what this does and does not skip	`claude --dangerously-skip-permissions`
`--debug`	Enable debug mode with optional category filtering (for example, `"api,hooks"` or `"!statsig,!file"`)	`claude --debug "api,mcp"`
`--debug-file <path>`	Write debug logs to a specific file path. Implicitly enables debug mode. Takes precedence over `CLAUDE_CODE_DEBUG_LOGS_DIR`	`claude --debug-file /tmp/claude-debug.log`
`--disable-slash-commands`	Disable all skills and commands for this session	`claude --disable-slash-commands`
`--disallowedTools`, `--disallowed-tools`	Deny rules. A bare tool name removes the matching tools from the model's context: `"Edit"` removes Edit, `"*"` removes every tool, and `"mcp__*"` removes every MCP tool. A scoped rule such as `Bash(rm *)` leaves the tool available and denies only matching calls	`"Bash(git log *)" "Bash(git diff *)" "Edit"`
`--effort`	Set the **effort level** for the current session. Options: `low`, `medium`, `high`, `xhigh`, `max`; available levels depend on the model. Overrides the `effortLevel` setting for this session and does not persist	`claude --effort high`
`--enable-auto-mode`	Removed in v2.1.111. Auto mode is now in the `Shift+Tab` cycle by default; use `--permission-mode auto` to start in it	`claude --permission-mode auto`
`--exclude-dynamic-system-prompt-sections`	Move per-machine sections from the system prompt (working directory, environment info, memory paths, git-repo flag) into the first user message. Improves prompt-cache reuse across different users and machines running the same task. Only applies with the default system prompt; ignored when `--system-prompt` or `--system-prompt-file` is set. Use with `-p` for scripted, multi-user workloads	`claude -p --exclude-dynamic-system-prompt-sections "query"`
`--exec`	Run a shell command as a PTY-backed background job instead of starting a Claude session. Use with `--bg` to launch from the shell	`claude --bg --exec 'pytest -x'`

Flag	Description	Example
`--fallback-model`	Enable automatic fallback to the specified model(s) when the primary model is overloaded or not available, for example a retired model. Accepts a comma-separated list tried in order. See **Fallback model chains**. To persist a chain across sessions, use the `fallbackModel` **setting**, which this flag overrides	`claude --fallback-model sonnet,haiku`
`--fork-session`	When resuming, create a new session ID instead of reusing the original (use with `--resume` or `--continue`)	`claude --resume abc123 --fork-session`
`--from-pr`	Resume sessions linked to a specific pull request. Accepts a PR number, a GitHub or GitHub Enterprise PR URL, a GitLab merge request URL, or a Bitbucket pull request URL. Sessions are linked automatically when Claude creates the pull request	`claude --from-pr 123`
`--ide`	Automatically connect to IDE on startup if exactly one valid IDE is available	`claude --ide`
`--init`	Run **Setup hooks** with the `init` matcher before the session (print mode only)	`claude -p --init "query"`
`--init-only`	Run **Setup** and `SessionStart` hooks, then exit without starting a conversation	`claude --init-only`
`--include-hook-events`	Include all hook lifecycle events in the output stream. Requires `--output-format stream-json`	`claude -p --output-format stream-json --verbose --include-hook-events "query"`
`--include-partial-messages`	Include partial streaming events in output. Requires `--print` and `--output-format stream-json`	`claude -p --output-format stream-json --verbose --include-partial-messages "query"`
`--input-format`	Specify input format for print mode (options: `text`, `stream-json`)	`claude -p --output-format json --input-format stream-json`
`--json-schema`	Get validated JSON output matching a JSON Schema after agent completes its workflow (print mode only, see **structured outputs**)	`claude -p --json-schema '{"type":"object","properties":{...}}' "query"`
`--maintenance`	Run **Setup hooks** with the `maintenance` matcher before the session (print mode only)	`claude -p --maintenance "query"`
`--max-budget-usd`	Maximum dollar amount to spend on API calls before stopping (print mode only)	`claude -p --max-budget-usd 5.00 "query"`

Flag	Description	Example
`--max-turns`	Limit the number of agentic turns (print mode only). Exits with an error when the limit is reached. No limit by default	`claude -p --max-turns 3 "query"`
`--mcp-config`	Load MCP servers from JSON files or strings (space-separated)	`claude --mcp-config ./mcp.json`
`--model`	Sets the model for the current session with an alias for the latest model (`sonnet`, `opus`, `haiku`, or `fable`) or a model's full name. Overrides the `model` setting and `ANTHROPIC_MODEL`	`claude --model claude-sonnet-4-6`
`--name`, `-n`	Set a display name for the session, shown in `/resume` and the terminal title. You can resume a named session with `claude --resume <name>`. `/rename` changes the name mid-session and also shows it on the prompt bar	`claude -n "my-feature-work"`
`--no-chrome`	Disable **Chrome browser integration** for this session	`claude --no-chrome`
`--no-session-persistence`	Disable session persistence so sessions are not saved to disk and cannot be resumed. Print mode only. The `CLAUDE_CODE_SKIP_PROMPT_HISTORY` environment variable does the same in any mode	`claude -p --no-session-persistence "query"`
`--output-format`	Specify output format for print mode (options: `text`, `json`, `stream-json`)	`claude -p "query" --output-format json`
`--permission-mode`	Begin in a specified **permission mode**. Accepts `default`, `acceptEdits`, `plan`, `auto`, `dontAsk`, or `bypassPermissions`. Overrides `defaultMode` from settings files	`claude --permission-mode plan`
`--permission-prompt-tool`	Specify an MCP tool to handle permission prompts in non-interactive mode	`claude -p --permission-prompt-tool mcp_auth_tool "query"`
`--plugin-dir`	Load a plugin from a directory or `.zip` archive for this session only. Each flag takes one path. Repeat the flag for multiple plugins: `--plugin-dir A --plugin-dir B.zip`	`claude --plugin-dir ./my-plugin`
`--plugin-url`	Fetch a plugin `.zip` archive from a URL for this session only. Repeat the flag for multiple plugins, or pass space-separated URLs in a single quoted value	`claude --plugin-url https://example.com/plugin.zip`
`--print`, `-p`	Print response without interactive mode (see **Agent SDK documentation** for programmatic usage details)	`claude -p "query"`
`--prompt-suggestions`	Emit a `prompt_suggestion` message after each turn with a predicted next user prompt. Requires `--print`, `--output-format stream-json`, and `--verbose`. See **Prompt suggestions**	`claude -p --prompt-suggestions --output-format stream-json --verbose "query"`
`--remote`	Create a new **web session** on claude.ai with the provided	`claude --remote "Fix`

Flag	Description	Example
	task description	`the login bug"`
`--remote-control`, `--rc`	Start an interactive session with **Remote Control** enabled so you can also control it from claude.ai or the Claude app. Optionally pass a name for the session	`claude --remote-control "My Project"`
`--remote-control-session-name-prefix <prefix>`	Prefix for auto-generated **Remote Control** session names when no explicit name is set. Defaults to your machine's hostname, producing names like `myhost-graceful-unicorn`. Set `CLAUDE_REMOTE_CONTROL_SESSION_NAME_PREFIX` for the same effect	`claude remote-control --remote-control-session-name-prefix dev-box`
`--replay-user-messages`	Re-emit user messages from stdin back on stdout for acknowledgment. Requires `--input-format stream-json` and `--output-format stream-json`	`claude -p --input-format stream-json --output-format stream-json --verbose --replay-user-messages`
`--resume`, `-r`	Resume a specific session by ID or name, or show an interactive picker to choose a session. The picker and name search include sessions that added this directory with `/add-dir`; passing a session ID searches only the current project directory and its git worktrees. As of v2.1.144, **background sessions** appear in the picker marked with `bg`	`claude --resume auth-refactor`
`--safe-mode`	Start with all customizations disabled to troubleshoot a broken configuration: CLAUDE.md, skills, plugins, hooks, MCP servers, custom commands and agents, output styles, workflows, custom themes, custom keybindings, status line and file-suggestion commands, LSP servers, and auto-memory do not load. Authentication, model selection, built-in tools, and permissions work normally, which differs from `--bare`. Managed settings policy still applies, including policy-configured hooks, status line, and file-suggestion commands; managed plugins, managed skills, managed CLAUDE.md, and policy-configured MCP servers do not. Useful for checking whether a customization is what triggers **automatic fallback from Fable 5**. Sets `CLAUDE_CODE_SAFE_MODE`	`claude --safe-mode`
`--session-id`	Use a specific session ID for the conversation (must be a valid UUID)	`claude --session-id "550e8400-e29b-41d4-a716-446655440000"`
`--setting-sources`	Comma-separated list of setting sources to load (`user`, `project`, `local`)	`claude --setting-sources user,project`
`--settings`	Path to a settings JSON file or an inline JSON string. Values you set here override the same keys in your `settings.json` files for this session. Keys you omit keep their file-based values. See **settings precedence**	`claude --settings ./settings.json`
`--strict-mcp-`	Only use MCP servers from `--mcp-config`, ignoring all	`claude --strict-mcp-`

Flag	Description	Example
`config`	other MCP configurations	`config --mcp-config ./mcp.json`
`--system-prompt`	Replace the entire system prompt with custom text	`claude --system-prompt "You are a Python expert"`
`--system-prompt-file`	Load system prompt from a file, replacing the default prompt	`claude --system-prompt-file ./custom-prompt.txt`
`--teleport`	Resume a **web session** in your local terminal	`claude --teleport`
`--teammate-mode`	Set how **agent team** teammates display: `in-process` (default), `auto`, `tmux`, or `iterm2` (added in v2.1.186). The default changed from `auto` in v2.1.179. Overrides the `teammateMode` setting for this session. See **Choose a display mode**	`claude --teammate-mode auto`
`--tmux`	Create a tmux session for the worktree. Requires `--worktree`. Uses iTerm2 native panes when available; pass `--tmux=classic` for traditional tmux	`claude -w feature-auth --tmux`
`--tools`	Restrict which built-in tools Claude can use. Use `""` to disable all, `"default"` for all, or tool names like `"Bash,Edit,Read"`. MCP tools are not affected; to deny those too, use `--disallowedTools "mcp__*"`, or pass `--strict-mcp-config` without `--mcp-config` so no MCP servers load	`claude --tools "Bash,Edit,Read"`
`--verbose`	Enable verbose logging, shows full turn-by-turn output. Overrides the `viewMode` setting for this session	`claude --verbose`
`--version`, `-v`	Output the version number	`claude -v`
`--worktree`, `-w`	Start Claude in an isolated **git worktree** at `<repo>/.claude/worktrees/<name>`. If no name is given, one is auto-generated. Pass `#<number>` or a GitHub pull request URL to fetch that PR from `origin` and branch the worktree from it	`claude -w feature-auth`

System prompt flags

Claude Code provides four flags for customizing the system prompt. All four work in both interactive and non-interactive modes.

Flag	Behavior	Example
`--system-prompt`	Replaces the entire default prompt	`claude --system-prompt "You are a Python expert"`
`--system-prompt-file`	Replaces with file contents	`claude --system-prompt-file ./prompts/review.txt`
`--append-system-prompt`	Appends to the default prompt	`claude --append-system-prompt "Always use TypeScript"`
`--append-system-prompt-file`	Appends file contents to the default prompt	`claude --append-system-prompt-file ./style-rules.txt`

`--system-prompt` and `--system-prompt-file` are mutually exclusive. The append flags can be combined with either replacement flag.

Choose based on whether Claude Code's default identity still fits your task. Use an append flag when Claude should remain a coding assistant that also follows your extra rules: per-invocation instructions, output formatting, or domain context for a `-p` script. Appending preserves the default tool guidance, safety instructions, and coding conventions, so you only supply what differs. Use a replacement flag when the surface, identity, or permission model differs from Claude Code's, like a non-coding agent in a pipeline that no human watches. Replacing drops all of the default prompt, including tool guidance and safety instructions, so you take responsibility for whatever your task still needs.

These flags apply only to the current invocation. For persistent personas you can switch between and share across a project, use **output styles**. For project conventions Claude should always follow, use **CLAUDE.md**. The **Agent SDK guide on system prompts** covers the same decision in more depth.

See also

- **Chrome extension** - Browser automation and web testing
- **Interactive mode** - Shortcuts, input modes, and interactive features
- **Quickstart guide** - Getting started with Claude Code
- **Common workflows** - Advanced workflows and patterns
- **Settings** - Configuration options
- **Agent SDK documentation** - Programmatic usage and integrations